TP3 — Tarefa 6: Pergunta de classificação

Dado um texto de atendimento ao cliente, qual é a intenção do usuário?

Este documento formaliza a pergunta que o Trimestre 2 vai resolver com aprendizado de máquina, complementando a Tarefa 1 (`regressao.ipynb`), que consolidou a fase analítica.

1. Definição da tarefa

Tipo de tarefa: classificação supervisionada **multiclasse** sobre texto livre (NLP).

Componente	Definição
Entrada `x`	Texto do ticket: `Ticket Subject` + `Ticket Description`
Saída `y`	A **intenção** do usuário — uma das 5 classes de `Ticket Type`
Função alvo	`f: texto → intenção`, aprendida de 8.469 exemplos rotulados
Exemplo	*"I'm having an issue with the GoPro Hero. Please assist."* → `Technical issue`

Valor de negócio. A intenção é o roteador do sistema: decide quem trata o ticket e alimenta o agente de IA do projeto. A API já antecipa essa decisão — o `/predict` (JWT, auditado no TP anterior) aguarda o modelo, e o enum `Intent` em `fastapi/models/predict.py` já define as 5 classes abaixo.

2. Classes (intenções)

#	Classe (`Ticket Type`)	Intenção do usuário	Exemplo canônico
1	`Technical issue`	Relatar defeito/falha e pedir correção	*"The screen freezes when I open the camera app."*
2	`Billing inquiry`	Dúvida sobre cobrança, fatura ou pagamento	*"I was charged twice for my subscription."*
3	`Product inquiry`	Pergunta sobre recursos ou compatibilidade	*"Does this model support 4K recording?"*
4	`Refund request`	Pedir devolução do valor pago	*"The item arrived damaged — I want my money back."*
5	`Cancellation request`	Pedir cancelamento de pedido/assinatura	*"Please cancel my order before it ships."*

As classes são **nominais** (sem ordem natural) e **mutuamente exclusivas**. Nota do EDA: os exemplos reais seguem templates sintéticos (≈69% em template fixo; placeholder `{product_purchased}` em 364 registros da versão limpa) — a normalização de texto é pré-requisito da modelagem.

3. Distribuição das classes no dataset

Versão limpa (`data/customer_support_tickets_clean.csv`, n = 8.469):

Classe	n	%
`Refund request`	1.752	20,7%
`Technical issue`	1.747	20,6%
`Cancellation request`	1.695	20,0%
`Product inquiry`	1.641	19,4%

Classe	n	%
`Billing inquiry`	1.634	19,3%

Consequências práticas: distribuição **balanceada** (19,3%–20,7%) → baseline aleatório ≈ **20%**, sem necessidade de rebalanceamento; métrica principal **F1 macro** (média das F1 por classe, com accuracy igualmente válida pelo equilíbrio); divisão treino/teste **estratificada**.

4. Por que aprendizado supervisionado — e por que não regressão linear

4.1 O problema pede ML supervisionado. Há 8.469 exemplos com a intenção conhecida, e a entrada é texto livre: alta dimensionalidade, paráfrases e variação lexical — nenhuma função escrita à mão cobre esse espaço, e regras de palavras-chave quebram na primeira paráfrase. ML supervisionado aprende `texto → P(classe | texto)` dos exemplos e generaliza.

4.2 A regressão linear não é adequada.

1. **Alvo nominal sem ordem.** Codificar intenções como números (1–5) inventa distâncias e ordem inexistentes; o erro "distância ao rótulo" não tem significado.
2. **Saída contínua não é probabilidade.** Nada garante $\hat{y} \in [0,1]$, soma 1 entre classes, nem uma decisão de fronteira bem definida.
3. **Fronteiras lineares insuficientes.** Em espaço TF-IDF de alta dimensão as classes raramente são separáveis por um hiperplano; o ferramental certo é a **regressão logística multinomial** (a "regressão" adequada a categorias), SVM, Naive Bayes ou ensembles.
4. **A Tarefa 1 mediu isso.** Com alvo contínuo, o modelo linear teve $R^2 \approx 0$ (η^2 máximo 0,001 e desempenho abaixo do baseline da média na validação cruzada): as features estruturadas não carregam sinal — o sinal está no **texto**, um problema de NLP supervisionado.

4.3 O que a Tarefa 1 lega ao modelo. (i) Ignorar `Ticket Subject` como preditor — o EDA provou independência com `Ticket Type` (H6, χ^2, $p = 0{,}981$); (ii) normalizar texto antes da vetorização (placeholders, templates); (iii) não confiar em metadados (prioridade, canal, idade).

5. Plano do Trimestre 2 (resumo)

1. **Pré-processamento:** lowercase, remoção de artefatos, tokenização; **TF-IDF** (uni e bigramas) sobre `Subject` + `Description`.
2. **Modelos:** baseline = regressão logística multinomial; candidatos = Complement Naive Bayes, SVM linear, gradient boosting (CV para hiperparâmetros).
3. **Avaliação:** holdout estratificado + CV 5-fold; accuracy, **F1 macro**, matriz de confusão; alvo: superar com folga os 20% do baseline aleatório.
4. **Implantação:** substituir o placeholder de `fastapi/routes/predict.py` pelo modelo treinado, preservando o contrato de segurança auditado (JWT, rate limit, `extra=forbid`).
5. **Riscos:** overfitting lexical em templates sintéticos → validação em amostra blindada e red team (etapa final do bloco).